AHB VIP Verification Project Report

Name: Mathan Gopal M

Introduction

The Advanced Microcontroller Bus Architecture (AMBA) AHB addresses the requirements of high-performance synthesizable designs. It is a bus interface that supports a single bus master and provides high-bandwidth operation.

The objective of this project is the development and verification of a highly robust, reusable, and parameterizable **AHB Verification IP (VIP)**. Rather than verifying a specific design block, this project focuses on verifying the AHB VIP itself by connecting an **AHB Master Universal Verification Component (UVC)** directly to an **AHB Slave Universal Verification Component (UVC)** in a back-to-back configuration under UVM (Universal Verification Methodology). This ensures that the VIP can generate, drive, monitor, and check any compliant AMBA 3 AHB bus traffic.

It implements the features required for high-performance, high-clock-frequency systems, including:

- Burst transfers
- Single-clock edge operation
- Non-tristate implementation
- Wide data bus configurations (e.g., 64, 128, 256, 512, and 1024 bits)

The most common AHB components are internal memory devices, external memory interfaces, high-bandwidth peripherals, and bridges.

Master and Slave Sides

According to the AHB protocol specifications, a typical system consists of Master and Slave components that interact through address/control and data phases:

AHB Master

An AHB Master provides address and control information to initiate read and write operations. The master drives the address bus (HADDR), the transfer type (HTRANS), burst type (HBURST), transfer size (HSIZE), and transfer direction (HWRITE). During a write operation, the master broadcasts data on the write data bus (HWDATA). The master relies on the slave's response signals (like HREADY and HRESP) to determine the success or extension of the pipelined data phase.

Master Interface Block Diagram

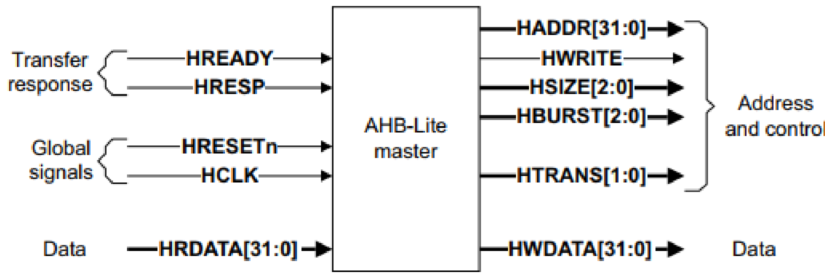


Figure Master interface

AHB Slave

An AHB Slave responds to the transfers initiated by the master in the system. The slave uses the HSELx select signal (typically provided by a system decoder) to control when it should respond to a bus transfer. The slave communicates the status of the data transfer back to the master indicating success (OKAY), failure (ERROR), or that it requires more time to complete the transaction by pulling the HREADYOUT signal LOW (inserting wait states). During a read operation, the slave provides the requested data on the HRDATA bus.

Slave Interface Block Diagram

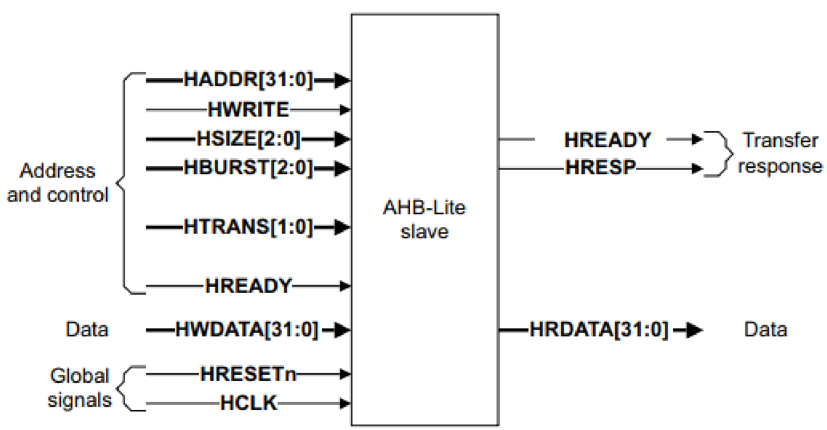


Figure Slave interface

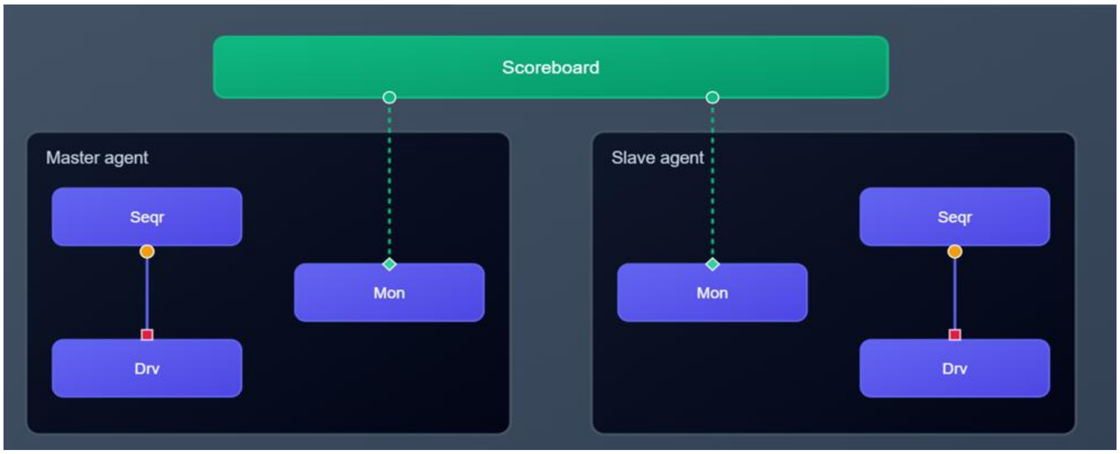
Signal Interface Map

Signal Name	Source	Direction	Description & Protocol Role
HCLK	System Clock	Input	System reference clock. All protocol events, signals, and interface samplings align strictly to the rising edge of HCLK.
HRESETn	Reset Controller	Input	Active-low system reset. Forces bus signals to their respective default protocol states.
HADDR [31:0]	Master Agent	Output	32-bit address bus. Drives the target memory address for the transfer.
HTRANS [1:0]	Master Agent	Output	Transaction type indicator. Valid states are: IDLE (00), BUSY (01), NONSEQ (10), and SEQ (11).
HSIZE [2:0]	Master Agent	Output	Indicates the size of individual data beats (Byte, Halfword, Word up to a maximum transfer size of 1024 bits).
HBURST [2:0]	Master Agent	Output	Specifies the burst type: Single, 4/8/16-beat Wrapping or Incrementing bursts.
HWRITE	Master Agent	Output	Direction indicator. Driven High for Write transfers, and Low for Read transfers.
HWDATA [31:0]	Master Agent	Output	Write data bus. Transfers data from Master to Slave during the data phase of write transfers.
HRDATA [31:0]	Slave Agent	Input	Read data bus. Transfers data from Slave to Master during the data phase of read transfers.
HREADY	Slave Agent	Input	Transfer readiness signal. Driven High to signal transaction completion. Driven Low to insert wait states.
HRESP	Slave Agent	Input	Transfer response status. Valid statuses: OKAY (0) or ERROR (1).

Description of the Scoreboard

The Scoreboard in this UVM environment acts as the central verification component that validates the protocol compliance, data integrity, and timing accuracy of the AHB Verification IP (VIP).

- **Data Collection:** It contains uvm_tlm_analysis_fifo components connected to the analysis ports of both the AHB Master Agent Monitor and the AHB Slave Agent Monitor to capture independent bus transaction streams from both sides of the bus.



- **Protocol & Data Integrity Checks:** It compares the address (HADDR), control signals (HWRITE, HSIZE, HBURST, HTRANS), and write data (HWDATA) driven by the Master Agent against the values sampled by the Slave Agent's monitor. This ensures that the transaction generated by the Master VIP is received exactly as-is by the Slave VIP with zero corruption or signal misalignment.
- **Read Data Loopback & Checking:** For read operations, the scoreboard ensures that the read data (HRDATA) and transfer response (HRESP) driven by the Slave Agent are correctly captured and read back by the Master Agent.
- **Burst Address and Wrap Verification:** It mathematically verifies that the address sequence for both incrementing bursts (INCR4, INCR8, INCR16) and wrapping bursts (WRAP4, WRAP8, WRAP16) conforms to the AMBA AHB specification, checking that wraps occur precisely on the computed boundary limits (e.g., 16-byte boundaries for 4-beat word wrapping bursts) and that no bursts cross 1KB boundaries.
- **Timing & Pipelining Validation:** It checks that the pipelined transition from the Address Phase to the Data Phase is handled correctly by both agents under various bus conditions, including wait states (HREADY) and two-cycle ERROR response scenarios.

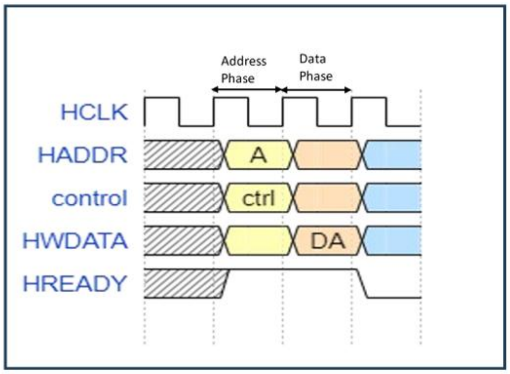
Waveforms

Below are the conceptual waveforms representing the AHB transfers, as adapted from the project reference documents.

1. Basic AHB Transfer

This demonstrates the standard pipelined operation where the Address phase is immediately followed by the Data phase.

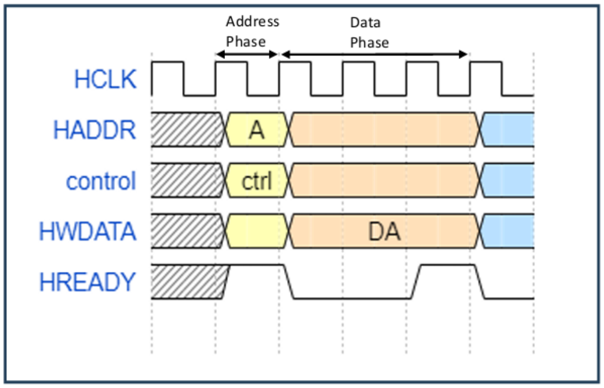
- **T0-T1 (Address Phase):** Master drives address A and control flags.
- **T1-T2 (Data Phase):** Slave samples address A and control flags on the rising edge of T1. The master drives write data Data (A) onto the bus, which is successfully sampled by the slave on the rising edge of T2 because HREADY is High.



2. Transfer with Wait State

This shows an AHB transfer where the slave requests extra time by pulling HREADY low, extending the data phase.

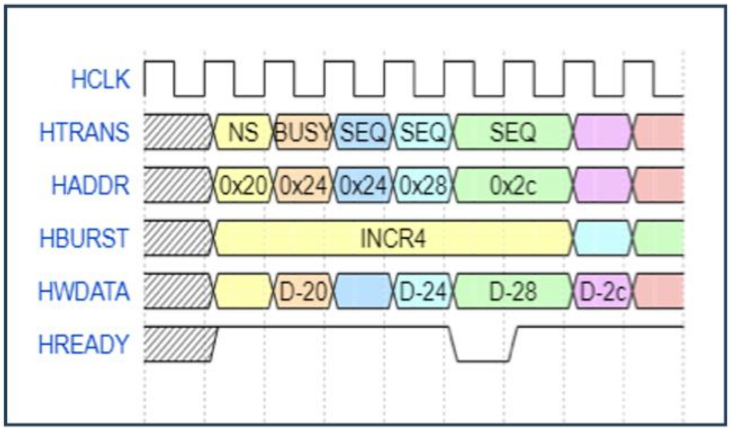
- **T1-T2 (Extended Data Phase):** Slave pulls HREADY Low to signal a wait state. The master must keep its address, control, and HWDATA completely stable across the extended cycles.
- **T2-T3 (Handshake Completed):** Slave drives HREADY High, allowing the write transfer to complete on the rising clock edge of T3.



3. Burst Transfer (Increment) - Pipelined Operation

A 4-beat incrementing burst (INCR4). The addresses increment sequentially.

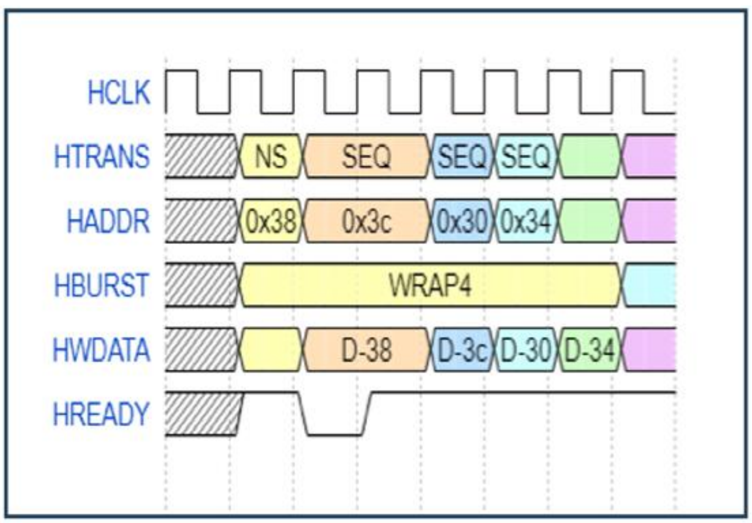
- **NONSEQ Cycle (0x20):** Initiates the first transfer of the burst.
- **BUSY Cycle (0x24):** Master inserts an idle cycle in the middle of the burst to request extra preparation time.
- **SEQ Cycles:** Sequential addresses are generated (0x24 -> 0x28 -> 0x2C) and transfer data beats (D-20 -> D-24 -> D-28 -> D-2C) are cleanly piped on subsequent cycles.



4. Burst Transfer (Wrapping) - Pipelined Operation

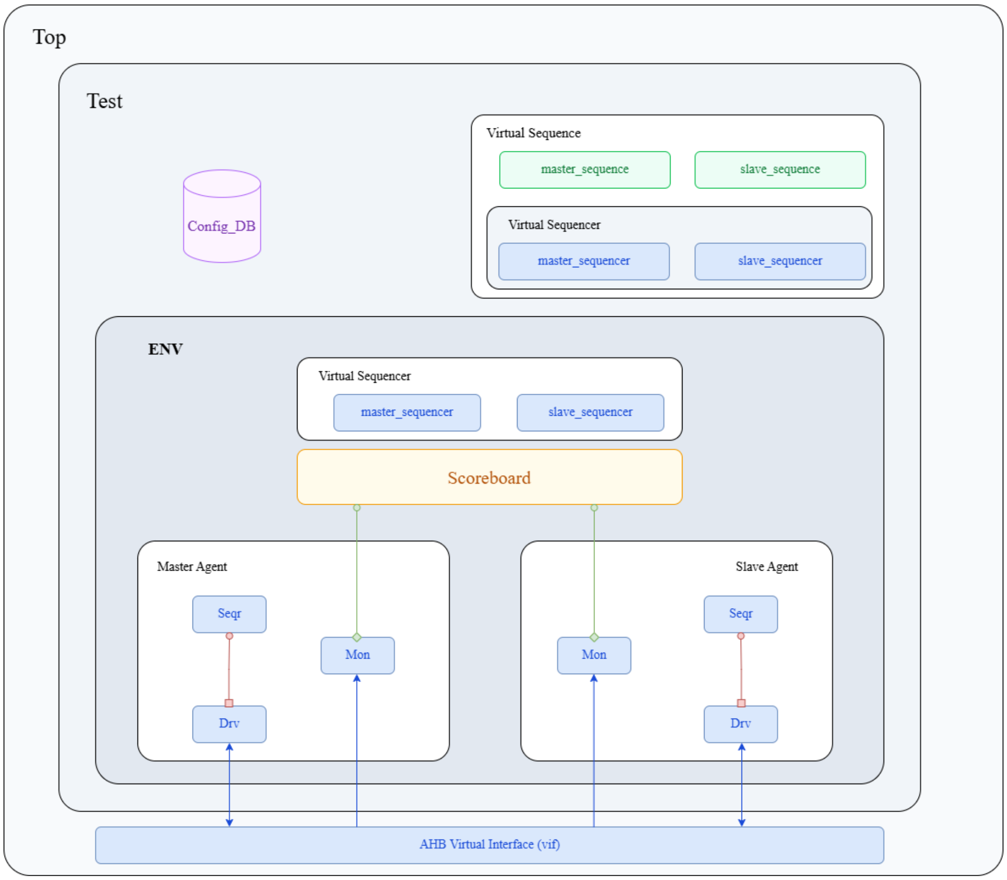
A 4-beat wrapping burst (WRAP4). Notice how the address wraps around the address boundary (e.g., from 0x3c back to 0x30).

- **Wrapping Boundary Calculation:** For a 4-beat word (4-byte) burst starting at 0x38, the wrap boundary is 4 x 4 bytes = 16 bytes.
- **Address Sequence:** The sequence increments up to the boundary (0x38 -> 0x3C), then wraps back symmetrically to the base boundary address block (0x30), before finishing at 0x34.



UVM TB Architecture

The UVM Testbench Architecture used to verify the AHB verification IP is shown in below figure.



Coverage Report

A key aspect of functional verification is measuring when testing is complete. The VIP was subjected to comprehensive regression testing monitored under the Coverage Engine. The final compiled results from the Questa Coverage Report are detailed below:

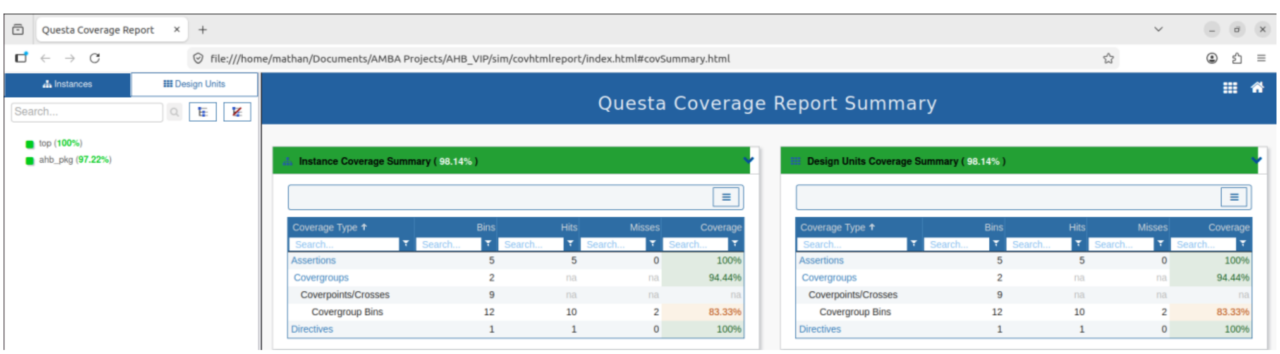


Fig: Overall Coverage Summary

Coverage Metrics Breakdown

Below is the summary of the coverage results obtained from the Questasim coverage database:

Covergroups type /ahb_pkg/ahb_scoreboard/mstr_covergroup

Summary	Bins	Hits
Coverpoints	9	8

coverpoints

Coverpoints	Bins	Hits	Misses	Goal	Coverage
Hwrite	1	1	0	100	100%
Hburst	1	1	0	100	100%
Htrans	2	2	0	100	100%
Hsize	3	2	1	100	66.66%
Haddr	1	1	0	100	100%
Hwdata	1	1	0	100	100%

Fig: Coverage for Master side Covergroup

Covergroups type /ahb_pkg/ahb_scoreboard/slv_covergroup

Summary	Bins	Hits
Coverpoints	3	3

coverpoints

Coverpoints	Bins	Hits	Misses	Goal	Coverage
Hrdata	1	1	0	100	100%
Hready	1	1	0	100	100%
Hresp	1	1	0	100	100%

Fig: Coverage for Slave side Covergroup

Takeaway from the Project and Difficulties Faced

Key Takeaways:

- **Protocol Mastery:** Gained a deep understanding of the AMBA AHB protocols, particularly the stark contrast between the pipelined, high-speed nature of AHB and the two-cycle
- **UVM Architecture:** Successfully structured a complex UVM testbench involving multiple agents, virtual sequences, and a robust scoreboard capable of handling cross-domain transaction comparisons.

Difficulties Faced:

- **Error Response:** If Error response is genreated from Slave Sequence, the difficulty were faced in the stopping of generation of the burst master, slave sequence.
- **Wait State Management:** Handling random HREADY insertions in the driver and ensuring the monitor only sampled valid data when HREADY was HIGH took significant debugging to get exactly right.
- **Scoreboard Synchronization:** Because an AHB burst (e.g., an 8-beat burst) is translated into 8 distinct transfers, ensuring the scoreboard correctly modeled this 1-to-N transaction relationship without dropping comparisons between Master monitor transaction & slave monitor transaction.
- **Wrapping Bursts Calculation:** Implementing the address calculation logic for WRAP4, WRAP8, and WRAP16 inside the testbench reference model to match the exact behavior of the design was mathematically tricky, especially when crossing boundary limits.